Documento de Arquitetura de Software

Chama Aí – Plataforma de Serviços do Interior

Autor: Daniel Victor Bezerra | Versão 1.0 | Macro Entrega 2 – Projeto e Arquitetura

1. Introdução

Este documento descreve a arquitetura de software do **Chama Aí**, plataforma web que conecta prestadores de serviço do interior (eletricistas, encanadores, pedreiros, babás, faxineiros, diaristas etc.) a contratantes, com perfil público, valores, disponibilidade e agendamento. A arquitetura parte dos artefatos da Macro Entrega 1 (Visão, Requisitos, Histórias, Backlog e Plano de IA) e orienta a implementação do MVP por uma equipe de uma pessoa.

Escopo: MVP com cadastro e login, perfil do prestador, envio e verificação de documento de identificação, busca, agendamento e contato por WhatsApp (RF01 a RF09, RF11, RF12, RF17 a RF19).

2. Direcionadores arquiteturais

Direcionador	Origem	Consequência na arquitetura
Equipe de uma pessoa e prazo curto	Visão do projeto	Monolito modular (um único back-end), poucas tecnologias, convenções padronizadas.
Baixo custo (RNF09)	Requisitos	Ferramentas gratuitas; hospedagem simples; sem serviços pagos obrigatórios.
Back-end em Java	Definição do projeto	Spring Boot 3 com Java 21.
Front-end gerado com Lovable (RNF10)	Plano de IA	Separação front-end/back-end por API REST; o back-end não depende do front-end.
Segurança e LGPD (RNF03, RNF04, RNF11)	Requisitos	Spring Security + JWT, BCrypt, armazenamento privado e acesso restrito aos documentos.
Usabilidade em celulares (RNF01, RNF02)	Requisitos	SPA responsiva, mobile first; respostas JSON enxutas para conexões lentas.
Manutenibilidade (RNF07)	Requisitos	Camadas bem definidas, pacotes por funcionalidade, migrações versionadas, testes nas regras de negócio.

3. Visão geral da arquitetura

Adotou-se o estilo **cliente-servidor com API REST**. O navegador executa uma aplicação de página única (SPA) em React, que se comunica por HTTPS/JSON com um back-end Spring Boot organizado como **monolito modular em camadas** (Web, Serviço e Infraestrutura). Os dados ficam em PostgreSQL e os documentos de identificação em armazenamento privado, acessado por uma interface (*ArmazenamentoArquivos*) que isola o local físico dos arquivos. O contato entre as partes usa links do WhatsApp (wa.me), sem integração de API.

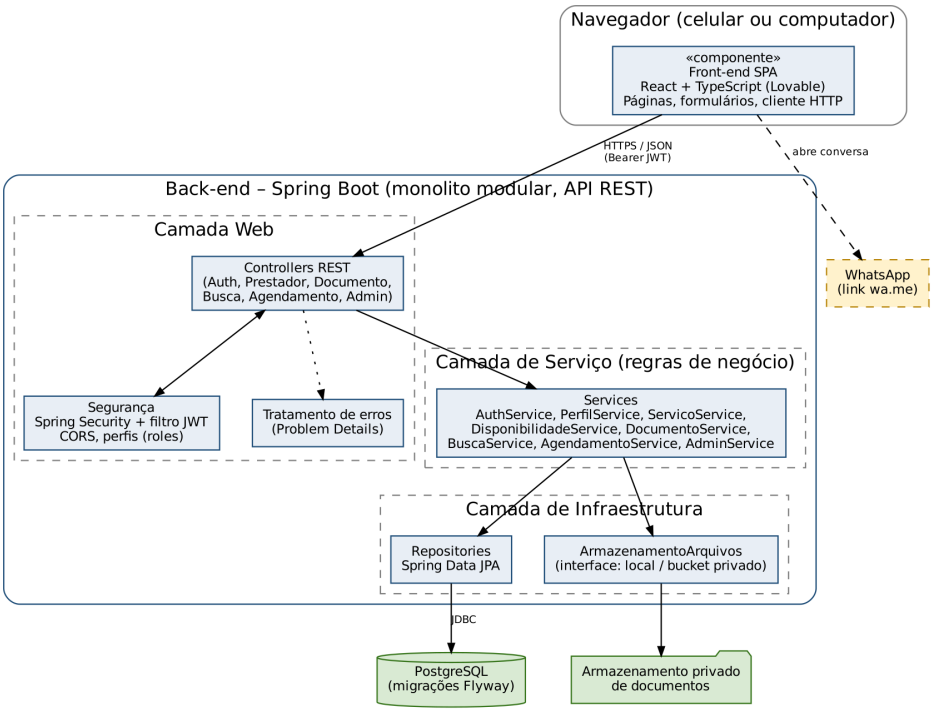


Figura 1 – Diagrama de componentes da arquitetura

4. Módulos do back-end

Módulo (pacote)	Responsabilidade	Requisitos	Entidades
auth	Cadastro, login, emissão e validação de JWT, perfis de acesso.	RF01, RF02, RNF03	Usuario
usuario	Dados do usuário e tipos (contratante, prestador, admin).	RF01	Usuario
prestador	Perfil profissional, serviços e disponibilidade; regra de publicação do perfil.	RF03 a RF05, RN02, RN03	PerfilPrestador, Servico, Disponibilidade
documento	Envio, armazenamento, verificação e status do documento de identificação.	RF17 a RF19, RN12 a RN16	DocumentoIdentificacao
busca (dentro de prestador)	Listagem e filtros públicos por categoria e cidade; página pública do perfil.	RF06, RF07	PerfilPrestador
agendamento	Solicitação, confirmação, recusa, cancelamento e conclusão; conflitos de horário.	RF08 a RF11, RF13, RN04 a RN08	Agendamento
catalogo	Categorias e cidades usadas em filtros e cadastros.	RF06, RF14	Categoria, Cidade
admin	Painel do administrador: documentos, categorias e desativação de perfis.	RF14, RF18	(usa os demais)
compartilhado	Segurança, CORS, tratamento de erros, utilitários e armazenamento de arquivos.	RNF03, RNF11	—

5. Camadas e organização do código

- **Web (Controllers):** recebem requisições HTTP, validam entrada (Bean Validation), chamam serviços e devolvem DTOs. Não contêm regras de negócio.
- **Serviço (Services):** concentram as regras de negócio (RN01 a RN16) e as transações.
- **Infraestrutura (Repositories, ArmazenamentoArquivos):** acesso a dados (Spring Data JPA) e a arquivos.
- **Pacote por funcionalidade:** cada módulo reúne seus controllers, services, repositories, entidades e DTOs, facilitando a evolução e o entendimento por uma única pessoa.
- **DTOs:** as entidades JPA nunca são expostas na API.

6. Tecnologias, frameworks e serviços

Camada	Tecnologia	Justificativa
Linguagem / plataforma	Java 21 (LTS)	Exigência do projeto; versão de suporte longo.
Framework back-end	Spring Boot 3.5 (Web, Validation, Data JPA, Security, Actuator)	Padrão de mercado, produtividade e ampla documentação.
Banco de dados	PostgreSQL 16	Gratuito, robusto; permite restrição de exclusão para evitar horários conflitantes (RN05).
Migrações	Flyway	Histórico versionado do esquema; reprodutível em qualquer ambiente.
Autenticação	Spring Security + JWT (jjwt) + BCrypt	Sem estado no servidor; adequado a SPA; senhas com hash.
Documentação da API	springdoc-openapi (Swagger UI)	Gera o contrato da API, também usado como base para os prompts do Lovable.
Front-end	React + TypeScript (gerado com Lovable, revisado com Claude)	Acelera o desenvolvimento por uma pessoa; interface responsiva.
Testes	JUnit 5, Mockito, Spring Boot Test, H2	Testes de regras de negócio e de endpoints.
Build e CI	Maven, GitHub Actions	Build reprodutível e testes automáticos a cada commit.
Contêineres	Docker e Docker Compose	Ambiente local igual ao de produção (banco e API).
Hospedagem	Render ou Railway (API), PostgreSQL gerenciado, front-end em Lovable/Vercel/Netlify	Camadas gratuitas ou de baixo custo.

7. Decisões arquiteturais

ID	Decisão	Justificativa	Alternativa descartada
AD-01	Monolito modular em camadas.	Menor complexidade de desenvolvimento, teste e implantação para uma pessoa.	Microsserviços: custo operacional desproporcional.
AD-02	SPA em React consumindo API REST.	Permite usar o Lovable e trocar o front-end no futuro (RNF10).	Thymeleaf renderizado no servidor: acopla interface e back-end.
AD-03	Lovable somente para o front-end; dados via API Java.	Mantém as regras de negócio e a segurança no back-end exigido pelo projeto.	Usar o back-end padrão do Lovable (Supabase): fugiria do requisito de back-end em Java.
AD-04	PostgreSQL com Flyway.	Integridade relacional e restrições no banco (inclusive RN05).	MongoDB: dados são relacionais.
AD-05	JWT no cabeçalho Authorization, expiração de 2 horas.	API sem estado e simples de consumir pela SPA.	Sessão em servidor: exige estado e afinidade de sessão.
AD-06	Documentos em armazenamento privado atrás da interface ArmazenamentoArquivos.	MVP em volume/disco privado; evolução para bucket privado sem alterar as regras.	Gravar o arquivo no banco: infla o banco e os backups.
AD-07	Contato por link do WhatsApp (wa.me).	Zero custo e familiar ao público; evita chat próprio.	API oficial do WhatsApp: custo e burocracia.
AD-08	Verificação manual dos documentos pelo administrador.	Simples e suficiente para o MVP; mantém o controle humano.	OCR/biometria facial: custo e complexidade fora do escopo.

8. Interfaces e integrações

A comunicação entre front-end e back-end usa **API REST em JSON** sob o prefixo */api*. Perfis de acesso: **Público** (sem login), **CONTRATANTE**, **PRESTADOR**, **ADMIN**. Listas têm paginação (*pagina* e *tamanho*). Erros seguem o formato Problem Details (RFC 7807), com status HTTP coerentes (400, 401, 403, 404, 409 para conflito de horário).

Método e rota	Acesso	Descrição	Req.
POST /api/auth/cadastro	Público	Cria usuário (contratante ou prestador).	RF01
POST /api/auth/login	Público	Autentica e retorna o JWT.	RF02
GET /api/auth/me	Logado	Dados do usuário autenticado.	RF02
GET /api/categorias · GET /api/cidades	Público	Listas para filtros e cadastros.	RF06
GET /api/prestadores?categoria=&cidade=&pagina=	Público	Busca prestadores publicados.	RF06
GET /api/prestadores/{id}	Público	Perfil público (serviços, valores, disponibilidade).	RF07
GET /api/prestadores/{id}/horarios?data=	Público	Horários livres do prestador na data.	RF08
GET · PUT /api/prestador/perfil	PRESTADOR	Consulta e edita o próprio perfil.	RF03
GET · POST · PUT · DELETE /api/prestador/servicos	PRESTADOR	Gerencia serviços e valores.	RF04

Método e rota	Acesso	Descrição	Req.
PUT /api/prestador/disponibilidade	PRESTADOR	Define dias e horários.	RF05
POST /api/prestador/documento (multipart)	PRESTADOR	Envia documento de identificação.	RF17
GET /api/prestador/documento/status	PRESTADOR	Status e motivo de rejeição.	RF19
POST /api/agendamentos	CONTRATANTE	Solicita agendamento (status PENDENTE).	RF08
GET /api/agendamentos	Logado	Lista os agendamentos do usuário.	RF11
PATCH /api/agendamentos/{id}/confirmar | recusar	PRESTADOR	Aceita ou recusa a solicitação.	RF09
PATCH /api/agendamentos/{id}/cancelar	Logado (parte)	Cancela antes da data.	RF10
PATCH /api/agendamentos/{id}/concluir	PRESTADOR	Marca como concluído.	RF13
GET /api/admin/documentos?status=	ADMIN	Lista documentos para análise.	RF18
GET /api/admin/documentos/{id}/arquivo	ADMIN	Baixa o arquivo para conferência.	RF18
PATCH /api/admin/documentos/{id}/aprovar | rejeitar	ADMIN	Aprova ou rejeita (motivo obrigatório).	RF18
CRUD /api/admin/categorias · PATCH /api/admin/usuarios/{id}/desativar	ADMIN	Moderação (v1.1).	RF14

Integrações externas: (1) **WhatsApp** – o front-end monta o link *https://wa.me/55DDDNUMERO?text=...* com mensagem pronta; não há chamada de API. (2) **Armazenamento de arquivos** – acessado pelo back-end por meio da interface *ArmazenamentoArquivos*. (3) **E-mail** – previsto para recuperação de senha (RF16), fora do MVP.

9. Segurança e LGPD

Tema	Medida
Autenticação	Senhas com BCrypt; JWT assinado com segredo vindo de variável de ambiente; expiração de 2 horas.
Autorização	Perfis PUBLICO, CONTRATANTE, PRESTADOR e ADMIN verificados no Spring Security; cada usuário só acessa os próprios agendamentos e perfil.
Entrada de dados	Bean Validation; consultas parametrizadas (JPA) contra SQL Injection; escape de saída no front-end contra XSS; CORS restrito ao domínio do front-end.
Upload de documentos	Somente JPG, PNG ou PDF até 5 MB, com verificação do tipo real do arquivo (não apenas da extensão); nome do arquivo trocado por UUID.
Armazenamento de documentos	Diretório ou bucket privado, fora de qualquer pasta pública; leitura apenas por ADMIN via endpoint autenticado; documento nunca aparece em perfil ou busca (RN16).
LGPD	Coleta mínima; consentimento explícito e registrado (consentimento_em); finalidade exclusiva de verificação de identidade; exclusão do arquivo após a verificação ou conforme a política de privacidade.
Segredos	Somente em variáveis de ambiente; arquivo .env fora do Git.

10. Implantação

O front-end é publicado como arquivos estáticos; a API roda em um contêiner Docker com o JAR do Spring Boot, conectado a um PostgreSQL gerenciado. Existem dois ambientes: **desenvolvimento** (Docker Compose local) e **produção** (hospedagem gratuita ou de baixo custo).

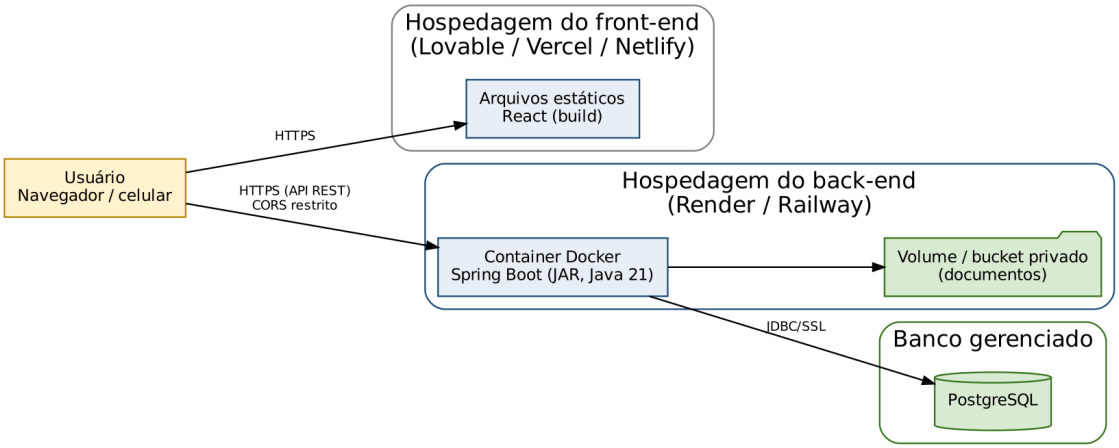


Figura 2 – Diagrama de implantação

11. Riscos e limitações técnicas

Risco / limitação	Prob.	Impacto	Mitigação
Documentos de identificação são dados pessoais sensíveis; vazamento gera dano legal e de reputação.	Média	Alto	Armazenamento privado, acesso só do ADMIN, HTTPS, exclusão após verificação, nunca usar documentos reais em testes ou IA.
Lovable gera integração própria (ex.: Supabase) ou código inconsistente com a API Java.	Alta	Médio	Prompts baseados no contrato OpenAPI, regra explícita de usar apenas a API REST, revisão do código e testes de integração.
Dois agendamentos no mesmo horário por requisições simultâneas.	Média	Médio	Validação no serviço e restrição de exclusão no PostgreSQL (RN05); responder 409 em caso de conflito.
Hospedagem gratuita com 'suspensão' por inatividade e armazenamento volátil.	Alta	Médio	Usar armazenamento persistente/bucket para documentos; avisar sobre a primeira resposta lenta; plano pago de baixo custo se necessário.
Verificação manual não detecta documento falso sofisticado e não escala.	Média	Médio	Conferência de legibilidade, validade e nome; possibilidade de denúncia e desativação; evolução futura com serviços de validação.
Escopo grande para uma pessoa (73 pontos no MVP).	Alta	Alto	MVP rígido; itens Should/Could adiados; front-end acelerado com IA.
Conexões lentas no interior.	Média	Médio	Páginas leves, imagens comprimidas, paginação e carregamento sob demanda.
Token JWT guardado no navegador pode ser exposto por XSS.	Baixa	Alto	Escape de saída, CSP no front-end, expiração curta; avaliar cookie HttpOnly em versão futura.
Sem pagamento, notificações push ou chat em tempo real (limitações do MVP).	—	Baixo	Previstos no backlog futuro; acerto financeiro e conversas feitos por WhatsApp.

12. Rastreabilidade requisitos – componentes

Requisitos	Componentes que os atendem
RF01, RF02, RNF03	módulo auth, SecurityConfig, filtro JWT
RF03 a RF05, RN03	módulo prestador (PerfilService, ServicoService, DisponibilidadeService)
RF06, RF07	módulo prestador (BuscaService), catalogo
RF08 a RF13, RN04 a RN08	módulo agendamento (AgendamentoService) e restrição ex_agendamento_sem_conflito
RF17 a RF19, RN12 a RN16, RNF11	módulo documento, ArmazenamentoArquivos, módulo admin
RF14	módulo admin, catalogo
RNF10	springdoc-openapi e contrato REST